

MVP

Aurélien Valdecasa

Objectif

Pouvoir reconnaître une personne à partir d'une photo de sa main. Pour ce faire, je dois construire un modèle de reconnaissance d'image afin d'importer la photo d'une main et savoir à quelle personne elle appartient dans mon jeu de données.

Tâches effectuées (ordre chronologique)

- **Recherche et choix de datasets (Kaggle)**

→ Choix principal : *Hands and Palm Dataset* (11 076 images). Justification : présence dos + paume, bon point de départ malgré une normalisation de la lumière, du fond et de la distance entre l'appareil photo et la main.

- **Prototype initial (TensorFlow)**

→ CNN basique (3 couches convolutives + 2 couches fully connected), entraîné sur CPU avec 216 images (8 classes) → 68 % d'accuracy.

Problème : lenteur CPU + problèmes GPU (drivers / compatibilité).

Décision : migrer vers PyTorch.

- **Réécriture et entraînement (PyTorch)**

→ Même architecture en PyTorch, 520 images (19 classes), validation croisée 5-fold → accuracy moyenne 82 % (max 87 %). Objectif : améliorer la robustesse de l'évaluation.

Problème : les données restent normalisées.

Décision : récolter mes propres données.

- **Collection de données personnelles**

→ Photos de 3 amis + moi-même (~20 photos par main) pour tester la variabilité réelle (angles, lumière, fonds). Utilisé pour évaluer la robustesse du modèle.

- **Évaluation sur jeu de test**

→ Petit test set de 18 images jamais vues → 100 %.

Problème : overfitting possible car on a très peu d'images.

Décision : chercher un plus gros jeu de données satisfaisant mes contraintes.

- **Recherche d'un dataset non normalisé**

→ Nouveau dataset Kaggle avec variations (distance, éclairage) : *Birjand University Mobile Palmprint Database (BMPD)*.

Problème : le fond diffère pour chaque personne mais pas pour chaque main.

- **Entraînement large (41 classes)**

→ 1 612 images, résultats stables, test : 319/323 = 98,76 %.

Problème : je crains que le modèle utilise le fond.

Décision : utiliser un modèle pour extraire un ROI (*Region Of Interest*) afin d'enlever le fond.

- **Extraction de ROI**

→ Tentative d'utiliser un repo GitHub (algorithme ROI).

Problème : dépendances obsolètes, mélange Python/C++, bugs.

Décision : développer un extracteur ROI avec la bibliothèque OpenCV.

- Tests successifs : segmentation par couleur → insuffisant ; par contraste → mieux ; combinaison couleur + contraste → encore sensible au seuil.
- Abandon de l'ajustement global de luminosité (n'a pas résolu le problème).
- Passage à dilatation + érosion (morphologie mathématique) → résultat satisfaisant (masque parfois débordant mais exploitable).

- **Entraînement après passage par ROI créant un masque noir pour éliminer le fond**
 - test : 81,11 %
 - Donc le modèle se servait bien du fond. Mais il reste performant.
- **Séparation mains droites / gauches pour évaluation**

Objectif : ne pas avoir deux mains différentes pour une seule personne.
- **Entraînement sur mains droites et gauches séparés + passage dans un ROI pour éliminer le fond**
 - test droite : 87,73 %
 - test gauche : 85,00%
 - Le modèle est légèrement meilleur. Cela confirme bien que le modèle est plus performant lorsqu'un label possède une seule main.
- **Amélioration de l'accessibilité**

→ Nettoyage du code, fusion des notebooks et commentaires. Début de conception d'une page web où l'on voit l'animation d'un clustering de mains.
- **Ajout d'un outil de recherche de mains similaires**

On peut maintenant entrer le chemin d'une image de main et voir les *k*-mains les plus similaires en utilisant les *features* des images données par le modèle.

Résultats

Dataset	Expériences	Nb images	Nb classes	Accuracy
Hands and Palm Images Dataset	Prototype TF (CPU)	216	8	68 %
Hands and Palm Images Dataset	Migration PyTorch (CV 5-fold)	520	19	moy. fold 82 %
Données personnelles (4 personnes, ~20/img chacun)	—	80	4	moy. fold 82 %, test 100 %
BMPD	—	1 612	41	test 98,76 %
BMPD	ROI	1 612	41	test 81,11 %
BMPD	Après ROI et séparé par côté	800 et 812	40 et 41	test mains gauche 85,00 %, test mains droite 87.73 %

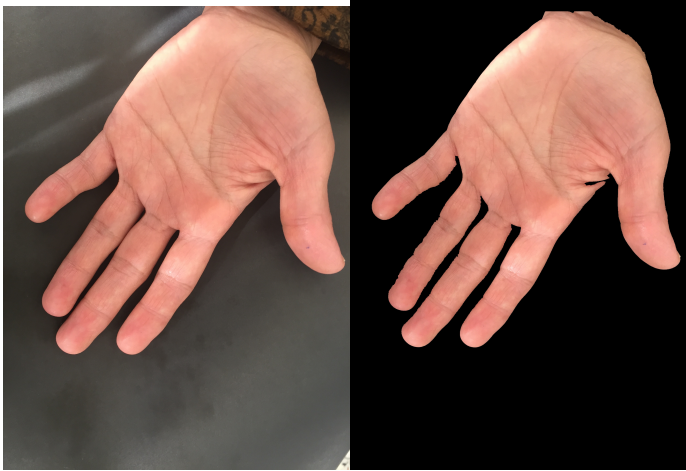
Perspectives

- **Améliorer le ROI** : remplacer l'approche heuristique par un modèle de segmentation léger pour réduire les débordements et détecter la main dans des scènes complexes. (Objectif : meilleure robustesse).
- **Augmenter la diversité des données** : photos prises à différents moments (ongles, bijoux, blessures), plus de sujets et de conditions d'éclairage. (Objectif : diminuer le risque d'overfitting et tester l'invariance dans le temps).
- **Détection de main dans une image générale** : image où l'objet principal n'est pas la main → détection de la main → détection ROI → classification.
- **Documenter** : créer une bibliographie répertoriant tous les outils utilisés et les articles sur lesquels je me suis appuyé. Rendre le projet plus accessible.

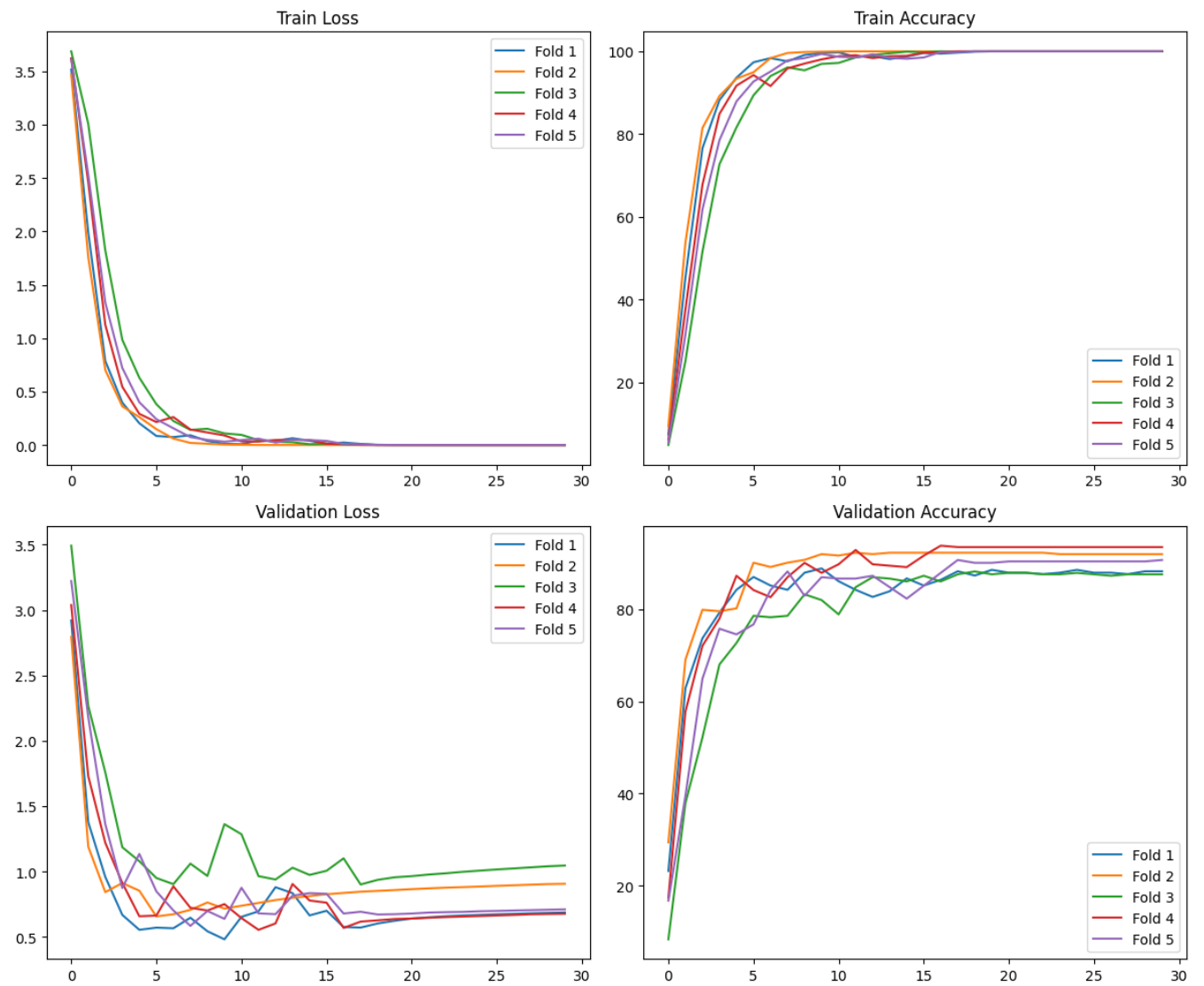
Livrables prêts

- Notebook Python avec ROI, entraînement du modèle et test du modèle.
- Site web contenant l'animation du clustering et le *devlog* (ouvrir avec Five server la page reco_mains.html).
- Les articles dont je me suis servi (une bibliographie complète est la prochaine étape de mon projet).

Annexes



- Exemple ROI :
- (1 612 images, 41 classes) : 98,76 % sur le test.



- Dataset après passage du ROI : 81,11 % sur le test :

